

SD Basic - Sockets

Talking TCP from SD BASIC, as a client and as a server.

SD BASIC can open a TCP connection, and it can listen for one. That is enough to call a web service, to feed a message queue, or to let another program talk to your application without going through a file.

SD folds case, so a program may be written in either case. Keywords are shown here in lower case. In the tables, *italics* mark something you supply and **bold** marks a word typed as it stands; braces mark an optional part.

One program can be both ends. `create.server.socket` calls `listen()`, so a client can connect on loopback and wait in the backlog until the same session accepts it.

The nine names

<code>create.server.socket(<i>addr</i> , <i>port</i> , <i>flags</i>)</code>	bind and listen. Returns a socket , or 0
<code>accept.socket.connection(<i>server</i> , <i>timeout</i>)</code>	take the next waiting connection
<code>open.socket(<i>addr</i> , <i>port</i> , <i>flags</i> { , <i>context</i> })</code>	dial out
<code>write.socket(<i>socket</i> , <i>data</i> , <i>flags</i> , <i>timeout</i>)</code>	returns the byte count
<code>read.socket(<i>socket</i> , <i>maxlen</i> , <i>flags</i> , <i>timeout</i>)</code>	returns the bytes read
<code>set.socket.mode(<i>socket</i> , <i>key</i> , <i>value</i>)</code>	returns 1 or 0
<code>socket.info(<i>socket</i> , <i>key</i>)</code>	ask about a socket
<code>close.socket <i>socket</i></code>	a statement , not a function
<code>server.addr(<i>name</i>)</code>	resolve a host name

The `SKT$` constants are in `SYSCOM KEYS.H`, reachable with `$include keys.h`. They are written out as numbers on this page so that what was passed is visible.

Opening the two ends

```
srv = create.server.socket('127.0.0.1', 45123, 0)
cli = open.socket('127.0.0.1', 45123, 0)
inc = accept.socket.connection(srv, 2000)
```

Flags `0` is TCP, stream. `0x00010000` selects UDP and `0x00020000` ICMP.

Both fail by returning zero, not by taking an `else` branch — they are functions, and there is no `then`/`else` on them. Test with `socket.info(s, 0)`, which is the one key that is safe to ask about a non-socket:

```
srv = create.server.socket('127.0.0.1', port, 0)
if socket.info(srv, 0) = 0 then
  crt 'cannot listen on ' : port : ', status ' : status()
  stop
end
```

How a connection fails:

a port nobody is listening on `socket.info 0, status()` **7005**, and SD **printed** *"Failed to connect: 111: Connection refused"* to the terminal by itself

a second listener on a port already in use `socket.info 0, status()` **7012**

The refusal message is printed whether you want it or not, which matters if you are polling for a service to come up.

What SOCKET.INFO tells you

KEY	MEANING	VALUE
<code>0</code>	is this a socket at all?	<code>1</code>
<code>1</code>	kind: 1 server, 2 incoming, 3 outgoing	see below
<code>2</code>	port	see below
<code>3</code>	IP address	<code>127.0.0.1</code>
<code>4</code>	blocking?	0 on every new socket

KEY	MEANING	VALUE
5	Nagle disabled?	0
6	keep-alive?	0
7	family: 1 IPv4, 2 IPv6	1

Key 2 means two different things and the difference is easy to miss. On the socket you dialled out with it is **the port you dialled**; on a socket you accepted it is **the far end's ephemeral port**:

	KIND	PORT
<code>create.server.socket(..., 45123, ...)</code>	1	45123
<code>open.socket(..., 45123, ...)</code>	3	45123
<code>accept.socket.connection(...)</code>	2	62602 — the client's own port, different every run

Reading and writing

```
n = write.socket(cli, 'hello world', 0, 2000)
d = read.socket(inc, 100, 0, 2000)
```

`write.socket` returned **11** for eleven bytes. `read.socket` returned the eleven bytes.

The timeout on a read does nothing unless the socket is blocking, and no socket starts out blocking. This is the trap on this page. On a socket with nothing sent to it:

THE CALL	WAITED	RETURNED
<code>read.socket(s, 100, 0, 5000)</code>	0 ms	nothing, <code>status()</code> 1011
<code>read.socket(s, 100, 1, 2000)</code>	2025 ms	nothing, <code>status()</code> 1011
after <code>set.socket.mode(s, 4, 1)</code> , <code>read.socket(s, 100, 0, 2000)</code>	2018 ms	nothing, <code>status()</code> 1011

A five second timeout was ignored completely. **Flag 1 is `SKT$BLOCKING` and flag 2 is `SKT$NON.BLOCKING`; flag 0 means "whatever the socket is set to", and a new socket is set to non-blocking.** Either pass `1` on every read that is allowed to wait, or call `set.socket.mode(s, 4, 1)` once after opening.

This is exactly the bug that passes its own tests. On loopback, and in any test where the reply is already sitting in the buffer, a non-blocking read returns the data immediately and everything looks right — **0 ms and five bytes**. Over a real network the reply arrives a few milliseconds later and the same code returns nothing.

`status()` is **1011**, timeout, for both "nothing yet" and "waited and gave up". It does not distinguish them.

TCP IS A STREAM, NOT A MESSAGE

Ten bytes written and read back in two pieces:

<code>read.socket(s, 4, ...)</code>	<code>abcd</code>
<code>read.socket(s, 100, ...)</code>	<code>efghij</code>

Nothing marks where one `write.socket` ended and the next began. **If your protocol has messages, put a length or a terminator in them yourself** and keep reading until you have a whole one.

THE BYTES ARRIVE EXACTLY AS SENT

	SENT	RECEIVED	IDENTICAL
field, value and subvalue marks	7	7	yes
<code>char(0)</code> , <code>char(1)</code> , <code>char(255)</code> and a letter	4	4	yes

A NUL does not terminate anything and a mark is not translated. A dynamic array can go down a socket and come back as itself.

SET.SOCKET.MODE

KEY	WHAT IT DOES	
<code>4</code>	blocking	set 0 → reads 0; set 1 → reads 1
<code>5</code>	disable Nagle	set 1 → reads 1
<code>6</code>	keep-alive	set 0 → reads 0; set 1 → reads 1
anything else	returns 0 , <code>status()</code> 1006	

Key 6 can be turned off as well as on: `set.socket.mode(s, 6, 0)` returns `1` and `socket.info(s, 6)` then reads `0`.

When the far end goes away

```
close.socket cli
```

There are two answers, not one, and which you get depends on which end closed. In the same session:

WHAT CLOSED

READING THE OTHER END GIVES

the **outgoing** socket closed, read the **accepted** one

nothing, `status()` **7013**

the **accepted** socket closed, read the **outgoing** one

nothing, `status()` **1008**

7013 is `ER_SKT_CLOSED` — `recv()` returned zero, an orderly shutdown. 1008 is `ER_FAILED` — `recv()` returned an error, and `os.error()` carries the operating system's code. **A read loop must treat both as the end of the conversation.** Testing only for 7013 is a loop that spins: that guard runs its full iteration count against a closed socket and comes back with nothing every time.

Neither is 1011. An idle socket gives 1011 and may still have something to say.

`close.socket` is a statement and it **refuses anything that is not a socket**, so do not call it on the zero that a failed `open.socket` returned.

Names and addresses

```
addr = server.addr('localhost')
```

`server.addr('127.0.0.1')`

`127.0.0.1`

`server.addr('localhost')`

`:::1`

`localhost` **RESOLVES TO THE IPv6 ADDRESS ON THIS PLATFORM.** A program that resolves `localhost` and then dials the answer is dialling IPv6, while `create.server.socket('127.0.0.1', ...)` is listening on IPv4 — family **1** — and the two do not meet. **Use `127.0.0.1` on both sides, or `:::1` on both sides, and do not resolve a name to get there.**

And a name that does not resolve blocks. `server.addr()` calls the operating system resolver with no timeout of its own. A call for a name that cannot resolve can still be waiting after **45 seconds**, with no way to abandon it short of abandoning the session. There is no way to bound it from BASIC. **Do not put `server.addr()` on a path where a user is waiting**, and prefer a configured address to a name.

A worked client, run rather than composed

This is the shape everything above adds up to. It was compiled and run with the far end in the same session, answering in **three** pieces so that the reassembly is exercised rather than assumed:

```
skt = open.socket(host, port, 0)
if socket.info(skt, 0) = 0 then
  crt 'no connection: ' : status()
  stop
end
x = set.socket.mode(skt, 4, 1)          ;* or every read returns at once
n = write.socket(skt, 'PING' : char(10), 0, 5000)

reply = ''
chunks = 0
loop
  chunk = read.socket(skt, 2048, 0, 5000)
  s = status()
  chunks += 1
  reply := chunk
until s = 7013 or s = 1008 or index(reply, char(10), 1) > 0 or chunks > 20
repeat
close.socket skt
```

Send `PON`, `G one` and a newline as three separate writes and the loop comes back with **one** chunk containing `PONG one`, the terminator at position 9 and `status()` **0**. The `chunks > 20` arm is not decoration — with the far end closed, the same loop guarded only by `s = 7013` runs every one of its iterations, because that end reports **1008**.

The terminator is yours, not TCP's. Three writes arrived as one read here and could just as easily arrive as three; the only reason the loop knows it is finished is the newline it went looking for.

What is not here

`writpkt` is a **restricted statement** — internal programs only. In an ordinary account it is "*Unrecognised statement*". It writes an SDClient protocol packet and is not a general socket write.

No TLS. There is nothing in SD BASIC that speaks HTTPS; a socket carries whatever bytes you put on it. Anything encrypted has to terminate outside SD.

The whole of this page is TCP. `0x00010000` and `0x00020000` are the UDP and ICMP flag values.

A server that serves more than one client at a time needs something to do the waiting, and SD BASIC has no `select` over several sockets. The shape that works is one accepted connection per session.

See also

[SD Basic - System and Environment](#) · [SD Basic - Sequential Files](#) · [SD Basic - String Functions](#) · [SD Basic - Locks and Transactions](#).